

CTeSP - Cursos Técnicos Superiores Profissionais

DESENVOLVIMENTO ÁGIL DE SOFTWARE

1ª Lista de Exercícios

Prof. Alex Maximilian Steil

Nome: André Filipe Manso Alves dos Santos Rosa Nº 2024520

Data: 31 / 07 / 2025 Turma: 1ºB

Git

Exercício 1: Configuração Inicial do Git

Objetivo: Configurar o Git com suas informações de utilizador.

1. Abra o terminal.
2. Configure seu nome de utilizador com o comando:

git config --global user.name "Seu Nome"

3. Configure seu e-mail com o comando:

git config --global user.email "[seu.email@exemplo.com](#)"

4. Verifique as configurações com:

git config --list

5. Pergunta: Qual é a importância de configurar nome e e-mail no Git?

Configurar o nome e o e-mail no git é importante para qualquer commit que o utilizador faça venha com o seu nome e e-mail. Com isto, o utilizador só se precisa de focar em fazer o commit, sem se preocupar com qual o nome ou o email a que está associado.

Exercício 2: Criando um Repositório Local

Objetivo: Criar e inicializar um repositório Git local.

1. *Crie uma diretoria chamada "**meu-projeto**" no seu computador.*

2. *Navegue até a diretoria usando o comando:*

cd caminho/para/meu-projeto

3. *Inicialize um repositório Git com:*

git init

4. *Verifique o status do repositório com:*

git status

5. *Pergunta: O que o comando git init faz? O que aparece no comando git status após inicializar o repositório?*

git init, como o nome indica, inicializa um repositório local, na localização onde o comando é executado.

git status, após a inicialização do repositório, mostra todos os ficheiros existentes na pasta do repositório, e mostra o estado dos ficheiros (staged/unstaged).

Caso exista um ficheiro .gitignore, todos os ficheiros que sejam abrangidos pelo .gitignore, são ignorados de todas as ações do git, incluindo o git status.

Exercício 3: Fazendo o Primeiro Commit

Objetivo: Adicionar ficheiros e criar um commit.

1. *Dentro da diretoria "**meu-projeto**", crie um ficheiro chamado "**index.html**" com o conteúdo: **Meu primeiro projeto***

echo Meu primeiro projeto > index.html

2. *Verifique o status do repositório com:*

git status

3. *Adicione o ficheiro à área de staging com:*

git add index.html

4. *Crie um commit com a mensagem:*

git commit -m "Adiciona ficheiro index.html inicial"

5. *Pergunta: Qual é a diferença entre git add e git commit?*

git add altera o estado dos ficheiros para staged, enquanto que o
git commit cria uma "snapshot" dos ficheiros staged.

O git add prepara os ficheiros para o commit, e o git commit guarda os ficheiros
staged numa "snapshot".

Exercício 4 – Histórico de Commits

Objetivo: Verificar o histórico dos commits executados

1. *Use o comando:*

git log

2. *Explore os commits e identifique o hash do primeiro commit*

3. *Filtre o log pelo autor*

git log –author="Meu nome"

4. *Comando Shortlog para identificar quais autores, quantos commits e quais foram*

git shortlog

Exercício 5: Visualizando Diferenças

Objetivo: Usar o comando git diff para comparar alterações.

1. *Modifique o ficheiro "index.html" adicionando um novo parágrafo.*

echo Fiz a primeira alteracao no ficheiro >> index.html

2. *Veja as diferenças antes de adicionar à área de staging com:*

git diff

3. *Adicione o ficheiro à área de staging e verifique novamente com:*

git diff

git diff --staged

4. *Pergunta: Qual é a diferença entre git diff e git diff --staged?*

git diff mostra ao utilizador todas as alterações feitas no repositório local, depois do último commit.

git diff --staged faz a mesma coisa, mas a flag "--staged" filtra só os ficheiros que estão staged.

Exercício 6: Desfazendo Alterações

Objetivo: Aprender a desfazer alterações no Git.

1. *Crie um novo ficheiro chamado "**teste.txt**" e adicione algum texto.*
2. *Adicione o ficheiro à área de staging com:*

git add teste.txt

3. *Desfaça a adição do ficheiro à área de staging com:*

git reset HEAD teste.txt

4. *Modifique o ficheiro **index.html** e, sem fazer commit, desfça as alterações com:*

git checkout index.html